

8.7.1.1 Consumo y Mapeo Avanzado de Procedimientos Almacenados en Hibernate y Spring Data JPA

Resumen

Esta lectura teórica aborda la integración de procedimientos almacenados (*Stored Procedures*) en arquitecturas empresariales Java utilizando **Hibernate**, **Jakarta Persistence (JPA)** y **Spring Data JPA**. Usted aprenderá a cartografiar llamadas a procedimientos almacenados nativos mediante la anotación `@Procedure`, a gestionar parámetros de entrada y salida (IN, OUT, INOUT, REF_CURSOR), a mapear conjuntos de resultados complejos (*ResultSets*) a entidades y DTOs mediante `@NamedStoredProcedureQuery`, y a sopesar los impactos de rendimiento y mantenibilidad al delegar lógica de negocio al motor de base de datos relacional.

Concepto Principal (El 'Qué')

Un **procedimiento almacenado** (*Stored Procedure*) es un conjunto de instrucciones SQL compiladas y almacenadas directamente en el motor de la base de datos relacional (tales como Microsoft SQL Server, PostgreSQL, Oracle o MySQL). A diferencia de las consultas ad-hoc o sintaxis JPQL/HQL enviadas en tiempo de ejecución, los procedimientos almacenados ejecutan planes de consulta precompilados en el servidor de base de datos.

En el contexto de **Hibernate** (como proveedor de la especificación JPA), la interacción con procedimientos almacenados se realiza mediante tres mecanismos principales:

1. La anotación `@Procedure` en Spring Data JPA

Es la alternativa más declarativa e idiomática en Spring Boot. Permite vincular un método dentro de una interfaz `@Repository` directamente a un procedimiento almacenado en la base de datos sin escribir código de infraestructura manual.

2. La anotación `@NamedStoredProcedureQuery` a nivel de Entidad

Definida dentro del modelo de dominio, esta anotación explicita el nombre del procedimiento almacenado, sus parámetros (`@StoredProcedureParameter`) con su respectivo `ParameterMode` (`IN`, `OUT`, `INOUT`, `REF_CURSOR`), y la clase de resultado o mapeo de conjunto de resultados (`@SqlResultSetMapping`).

3. La interfaz `StoredProcedureQuery` de la API `EntityManager`

Mecanismo de bajo nivel provisto por JPA (`jakarta.persistence.StoredProcedureQuery`) que ofrece control dinámico en tiempo de ejecución para registrar parámetros, ejecutar llamadas y extraer valores de retorno o múltiples `ResultSet`.

Tipos de Parámetros en Procedimientos Almacenados

- **`ParameterMode.IN`**: Parámetro de entrada enviado desde la aplicación Java hacia el procedimiento.
- **`ParameterMode.OUT`**: Parámetro de salida que el procedimiento calcula y retorna hacia Java.
- **`ParameterMode.INOUT`**: Parámetro que envía un valor inicial y retorna un valor modificado por el procedimiento.
- **`ParameterMode.REF_CURSOR`**: Puntero a un conjunto de datos en motores como Oracle o PostgreSQL para iterar filas de resultados.

Analogía (La Intuición)

Imagine que su aplicación Java es el mostrador de un taller de carpintería a la medida, y la base de datos es el taller especializado ubicado en el sótano.

Cuando su aplicación ejecuta una consulta SQL ad-hoc (o HQL), es el equivalente a enviar al carpintero una lista de instrucciones individuales paso a paso: *"corte la madera a 10 cm, lije el borde, aplique barniz y ensamble"*. El carpintero debe leer y planificar el proceso en cada pedido.

En cambio, invocar un **procedimiento almacenado** es equivalente a presionar el botón de una **máquina industrial preconfigurada** en el sótano: usted solo ingresa la materia prima por una

ranura (parámetros IN), la máquina ejecuta toda la secuencia de cortes y ensamblaje internamente a máxima velocidad sin consultar al mostrador, y finalmente entrega el producto terminado por una bandeja (parámetros OUT o ResultSet). El mostrador no necesita saber cómo operan los engranes internos de la máquina; solo envía la orden y recibe el resultado.

Ejemplo de Código e Implementación (El 'Cómo')

A continuación se presenta un escenario empresarial completo: la gestión de un catálogo de empleados y departamentos en Microsoft SQL Server / PostgreSQL consumido desde una aplicación Spring Boot 3.x con Java 21.

1. Script de la Base de Datos (SQL Server)

```
-- Creación de la tabla de empleados
CREATE TABLE Empleado (
    id BIGINT IDENTITY(1,1) PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    apellido VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    salario DECIMAL(12,2) NOT NULL,
    departamento VARCHAR(50) NOT NULL,
    activo BIT NOT NULL DEFAULT 1
);

GO

-- Procedimiento 1: Obtener lista de empleados por departamento (ResultSet)
CREATE PROCEDURE sp_obtener_empleados_por_departamento
    @p_departamento VARCHAR(50)
AS
BEGIN
    SET NOCOUNT ON;
    SELECT id, nombre, apellido, email, salario, departamento, activo
    FROM Empleado
    WHERE departamento = @p_departamento AND activo = 1
    ORDER BY apellido, nombre;
END;

GO

-- Procedimiento 2: Calcular métricas salariales (Parámetros OUT)
```

```

CREATE PROCEDURE sp_calcular_metricas_salario
    @p_departamento VARCHAR(50),
    @p_total_empleados INT OUTPUT,
    @p_promedio_salario DECIMAL(12,2) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        @p_total_empleados = COUNT(*),
        @p_promedio_salario = ISNULL(AVG(salario), 0.00)
    FROM Empleado
    WHERE departamento = @p_departamento AND activo = 1;
END;
GO

```

2. Mapeo de la Entidad JPA con `@NamedStoredProcedureQuery`

```

package cr.ac.ucr.ie.domain;

import jakarta.persistence.*;
import java.math.BigDecimal;

@Entity
@Table(name = "Empleado")
@NamedStoredProcedureQuery(
    name = "Empleado.spCalcularMetricasSalario",
    procedureName = "sp_calcular_metricas_salario",
    parameters = {
        @StoredProcedureParameter(mode = ParameterMode.IN, name = "p_departamento", type = String.class),
        @StoredProcedureParameter(mode = ParameterMode.OUT, name = "p_total_empleados", type = Integer.class),
        @StoredProcedureParameter(mode = ParameterMode.OUT, name = "p_promedio_salario", type = BigDecimal.class)
    }
)
public class Empleado {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String nombre;

    @Column(nullable = false, length = 100)

```

```

    private String apellido;

    @Column(nullable = false, length = 150, unique = true)
    private String email;

    @Column(nullable = false, precision = 12, scale = 2)
    private BigDecimal salario;

    @Column(nullable = false, length = 50)
    private String departamento;

    @Column(nullable = false)
    private Boolean activo;

    public Empleado() {}

    // Constructores, Getters y Setters omitidos por brevedad
}

```

3. Interfaz de Repositorio de Spring Data JPA (@Repository)

```

package cr.ac.ucr.ie.data;

import cr.ac.ucr.ie.domain.Empleado;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.query.Procedure;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.math.BigDecimal;
import java.util.List;
import java.util.Map;

@Repository
public interface EmpleadoRepository extends JpaRepository<Empleado, Long> {

    // 1. Invocación implícita por nombre de procedimiento coincidente en la BD
    @Procedure(procedureName = "sp_obtener_empleados_por_departamento")
    List<Empleado> obtenerEmpleadosPorDepartamento(@Param("p_departamento") String departamento);

    // 2. Invocación de NamedStoredProcedureQuery definido en la entidad
    @Procedure(name = "Empleado.spCalcularMetricasSalario")
    Map<String, Object> spCalcularMetricasSalario(@Param("p_departamento") String departamento);
}

```

4. Capa de Servicio de Negocio e Invocación Manual con EntityManager

```
package cr.ac.ucr.ie.business;

import cr.ac.ucr.ie.data.EmpleadoRepository;
import cr.ac.ucr.ie.domain.Empleado;
import jakarta.persistence.EntityManager;
import jakarta.persistence.ParameterMode;
import jakarta.persistence.PersistenceContext;
import jakarta.persistence.StoredProcedureQuery;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

@Service
public class EmpleadoService {

    private final EmpleadoRepository empleadoRepository;

    @PersistenceContext
    private EntityManager entityManager;

    public EmpleadoService(EmpleadoRepository empleadoRepository) {
        this.empleadoRepository = empleadoRepository;
    }

    @Transactional(readOnly = true)
    public List<Empleado> listarPorDepartamento(String departamento) {
        return empleadoRepository.obtenerEmpleadosPorDepartamento(departamento);
    }

    @Transactional(readOnly = true)
    public Map<String, Object> obtenerMetricasViaRepository(String departamento) {
        return empleadoRepository.spCalcularMetricasSalario(departamento);
    }

    @Transactional(readOnly = true)
    public Map<String, Object> obtenerMetricasDinamicasViaEntityManager(String departamento) {
        StoredProcedureQuery query = entityManager.createStoredProcedureQuery("sp_calcular_metricas_");

        // Registro explícito de parámetros de entrada y salida
        query.registerStoredProcedureParameter("p_departamento", String.class, ParameterMode.IN);
    }
}
```

```

        query.registerStoredProcedureParameter("p_total_empleados", Integer.class, ParameterMode.OUT);
        query.registerStoredProcedureParameter("p_promedio_salario", BigDecimal.class, ParameterMode.OUT);
    }

    // Asignación del parámetro IN
    query.setParameter("p_departamento", departamento);

    // Ejecución del procedimiento
    query.execute();

    // Extracción de parámetros OUT
    Integer total = (Integer) query.getOutputParameterValue("p_total_empleados");
    BigDecimal promedio = (BigDecimal) query.getOutputParameterValue("p_promedio_salario");

    Map<String, Object> resultado = new HashMap<>();
    resultado.put("totalEmpleados", total != null ? total : 0);
    resultado.put("promedioSalario", promedio != null ? promedio : BigDecimal.ZERO);

    return resultado;
}
}

```

Trampas Comunes

1. Olvidar la Anotación `@Transactional`

Las llamadas a procedimientos almacenados que leen o modifican datos (especialmente aquellos que manipulan *cursors* en Oracle o PostgreSQL) requieren ejecutarse obligatoriamente dentro de un contexto transaccional activo. Omitir `@Transactional` provocará una excepción `TransactionRequiredException` o la pérdida de conexión del cursor.

2. Desajuste de Nombres y Tipos de Parámetros (`ParameterMode`)

Si el nombre del parámetro declarado en `@StoredProcedureParameter` o en `query.registerStoredProcedureParameter()` no coincide exactamente con la firma del procedimiento en SQL Server/PostgreSQL, Hibernate lanzará una excepción `IllegalArgumentException` o `SQLException` indicando que el procedimiento esperaba parámetros no provistos.

3. Diferencias dialectales en el manejo de `REF_CURSOR`

En Oracle o PostgreSQL, la devolución de múltiples filas requiere declarar un parámetro `ParameterMode.REF_CURSOR`. Sin embargo, en SQL Server o MySQL los procedimientos retornan los conjuntos de datos directamente como instrucciones `SELECT` sin declarar un parámetro de salida adicional. Intentar registrar un `REF_CURSOR` en SQL Server causará un fallo de compilación SQL.

4. Acoplamiento Severo y Pérdida de Portabilidad

Abusar de la lógica de negocio en procedimientos almacenados rompe la independencia del motor de base de datos que ofrece JPA/Hibernate. Migrar de SQL Server a PostgreSQL requerirá reescribir todos los procedimientos almacenados en T-SQL a PL/pgSQL.

Pregunta de Reflexión

Suponga que un procedimiento almacenado realiza modificaciones masivas en varias tablas y retorna un conteo de filas afectadas mediante un parámetro `OUT`. Si una excepción de negocio ocurre en la capa de servicio Java *posterior* a la llamada del procedimiento pero *antes* de finalizar la transacción de Spring (`@Transactional`), ¿qué sucede con los cambios físicos realizados por el procedimiento en la base de datos? Justifique su respuesta basándose en el comportamiento de las transacciones ACID relacionales y el contexto de persistencia de Hibernate.